1. Senkron ve Asenkron Kod

Senkron Kod

Senkron Kodlar Sırayla ve Hemen Çalışır. Bir Satır Tamamlanmadan Sonraki Satıra Geçilmez.

```
print('A');
print('B');
print('C');
```

Çıktı:

```
A
B
C
```

Asenkron Kod

Asenkron İşlemler Zaman Alan Görevler İçin Kullanılır. Örneğin:

- API'den Veri Çekmek,
- Firebase İşlemleri,
- Dosya Okumak veya Yazmak,
- Veritabanı Sorgusu,
- Ağ İsteği Göndermek,
- Belirli Bir Süre Beklemek.

Örnek Düşünce:

- **Senkron:** Önce A Tamamen Biter, Sonra B Başlar.
- **Asenkron:** A ve B Başlatılır; Hangisi Önce Biterse Sonucu Önce Gelir.

2. Dart Kod Çalışma Sırası

```
Senkron Kod
   ↓
Microtask Queue
   ↓
Event Queue
```

2.1 Microtask Queue

Microtask Kuyruğu Yüksek Önceliklidir. Senkron Kod Tamamlandıktan Sonra Event Queue'ya Geçmeden Önce Çalıştırılır.

Microtask Oluşturma Yolları:

```dart
scheduleMicrotask(() {
  print('Microtask çalıştı');
});

Future.microtask(() {
  print('Future.microtask çalıştı');
});
```

2.2 Event Queue

Normal `Future` ve Timer Tabanlı İşlemler Event Queue'ya Eklenir.

```dart
Future(() {
  print('Normal Future Çalıştı.');
});

Future.delayed(Duration.zero, () {
  print('Timer Tabanlı Görev Çalıştı.');
});
```

Örnek: Çalışma Sırası

```dart
import 'dart:async';

void main() {
  print('1. Senkron Başladı.');

  scheduleMicrotask(() {
    print('2. Microtask Çalıştı.');
  });

  Future.delayed(Duration.zero, () {
    print('3. Future.delayed Çalıştı.');
  });

  Future(() {
    print('4. Normal Future Çalıştı.');

    scheduleMicrotask(() {
    print('5. Normal Future içindeki Microtask');
  });
  });

  Future.microtask(() {
    print('6. Future.microtask Çalıştı.');
  });

  Future(() {
    print('7. Diğer Future');
});

  print('8. Senkron Kod Tamamlandı.');
}
```

Çıktı

1. Senkron Başladı.
2. Senkron Kod Tamamlandı.
3. Microtask Çalıştı.
4. Future.microtask Çalıştı.
5. Future.delayed Çalıştı.

6. Normal Future Çalıştı.
7. Normal Future içindeki Microtask
8. Diğer Future

3. Future

Gelecekte **Bir Kez** Tamamlanacak Olan İşlemi Temsil Eder.

Bir Future Üç Durumda Olabilir:

1. **Devam Ediyor:** İşlem Henüz Tamamlanmadı.
2. **Başarılı:** Bir Değer Döndü.
3. **Başarısız:** Hata Oluştu.

Örnek:

```
Future<String> getUserName() async {
  await Future.delayed(const Duration(seconds: 1));
  return 'Mustafa';
}
```

Bu İşlem Yaklaşık Bir Saniye Sonra `String` Türünde Sonuç Döndürür.

Kullanımı:

```
void main() async {
  final userName = await getUserName();

  print(userName);
}
```

`async` ve `await`

- `async` : Bir Fonksiyonun Asenkron İşlem Yapacağını Belirtir.

- `await` : Future Tamamlanana Kadar O Fonksiyonun Devamındaki Kodu Bekletir.

> `await` Sadece `async` Olarak İşaretlenmiş Fonksiyonlarda Kullanılabilir.

`await` Yerine `then()` de Kullanılabilir:

Yeni Başlayanlar İçin Çoğu Durumda `async` / `await` Daha Okunaklıdır.

```dart
import 'dart:async';

Future<String> getUserName() async {
  // API'den Veri Geliyormuş Gibi 2 Saniye Bekler.
  await Future.delayed(const Duration(seconds: 2));

  // 2 Saniye Sonra Kullanıcı Adını Döndürür.
  return 'Mustafa';
}

Future<void> loadProfile() async {
  // Hemen Çalışır.
  print('1. Profil Yükleniyor...');

  // getUserName() Tamamlanınca .then() İçindeki Kod Çalışır.
  // Ardından Kullanıcı Adı userName Değişkenine Aktarılır.
  final userName = await getUserName().then((userName) {
    print('2. Kullanıcı Adı Başarıyla Alındı.');
    return userName;
  });

  // Kullanıcı Adı Geldikten Çalışır.
  print('3. Hoş Geldin, $userName');
}

void main() async {
  // Program İlk Olarak Buradan Başlar.
  print('0. Uygulama Başladı.');

  // loadProfile() Bitene Kadar main İçindeki Sonraki Satırı
Bekletir.
  await loadProfile();

  // loadProfile() Bittikten Sonra Çalışır.
  print('4. Profil İşlemi Tamamlandı.');
}
```

Çıktı

0. Uygulama Başladı.

1. Profil Yükleniyor...
2. Kullanıcı Adı Başarıyla Alındı.
3. Hoş Geldin, Mustafa
4. Profil İşlemi Tamamlandı.

Future Hata Yönetimi

```
Future<void> loadData() async {
  try {
    final data = await getUserName();
    print(data);
  } catch (error) {
    print('Bir Hata Oluştu: $error');
  }
}
```

4. `Future.wait()` — Paralel Veri İstekleri

Bir Ekranda Birden Fazla Bağımsız Veri Gerekiyorsa İşlemleri Tek Tek Bekletmek Yerine Aynı Anda Başlatmak Daha Verimli Olabilir.

Örnek Bir Dashboard Ekranında Şunlar Gerekebilir:

- Kullanıcı Profili,
- Hesap Bakiyesi,
- Bildirimler.

```
Future<void> fetchDashboardData() async {
  try {
    final results = await Future.wait([
      fetchUserProfile(),
      fetchAccountBalance(),
      fetchNotifications(),
    ]);

    print('Tüm Veriler Geldi: $results');
  } catch (error) {
    print('İsteklerden Biri Başarısız Oldu: $error');
  }
}
```

Bu Yapıda:

- Üç İstek Birlikte Başlatılır.
- `await` , Tüm Sonuçların Tamamlanmasını Bekler.
- İşlemlerden Biri Hata Verirse `catch` Bloğu Çalışır.
- Sonuçlar, `Future.wait` Listesine Yazılan Sırayla Döner.

> `Future.wait()` Sadece Birbirinden Bağımsız İşler İçin Uygundur. Bir İşlem Diğerinin Sonucuna İhtiyaç Duyuyorsa Sırayla `await` Kullanılmalıdır.

5. Stream

`Stream` , Zaman İçinde **Birden Fazla Veri** Üreten Akıştır.

Özellik	Future	Stream
Veri Sayısı	Tek Sonuç	Birden Fazla Sonuç
Süre	Bir Kez Tamamlanır	Zaman İçinde Devam Eder
Sonuç	Değer veya Hata Döner, Biter	Veri, Hata Ve Tamamlanma Olayları Gönderebilir
Örnek	Kullanıcı Profilini Çekmek	GPS Konumunu Sürekli Dinlemek
Benzetme	Tek Seferlik Sipariş	Açık Musluk / Boru Hattı

Stream Ne Zaman Kullanılır?

- Firebase Canlı Veri Dinleme,
- Canlı Borsa Verileri,
- GPS Konum Güncellemeleri,
- WebSocket Mesajları,
- Kullanıcı Girişlerini veya Olayları Dinleme.

StreamController ve Stream

`StreamController` , Stream'e Veri Göndermeyi ve Akışı Yönetmeyi Sağlar.

```dart
import 'dart:async';

void main() {
  final controller = StreamController<int>();

  final subscription = controller.stream.listen(
    (data) {
      print('Yeni Veri: $data');
    },
    onError: (error) {
      print('Hata Oluştu: $error');
    },
    onDone: () {
      print('Stream Tamamlandı.');
    },
  );

  controller.add(10);
  controller.add(20);
  controller.add(30);

  controller.close();
}
```

Çıktı:

```
Yeni Veri: 10
Yeni Veri: 20
Yeni Veri: 30
Stream Tamamlandı.
```

Kod	Görevi
`StreamController<int>()`	`int` Tipinde Veri Gönderecek Bir Controller Oluşturur.
`controller.stream`	Dinlenebilir Veri Akışıdır.
`listen()`	Stream'i Dinlemeye Başlar.
`controller.add()`	Stream'e Veri Ekler.
`onError`	Akış Hata Gönderirse Çalışır.
`onDone`	Stream Kapatıldığında Çalışır.
`controller.close()`	Stream'e Yeni Veri Göndermeyi Bitirir.
`subscription.cancel()`	Dinlemeyi Durdurur.

Single-Subscription Stream

Varsayılan `StreamController` , Genellikle **Tek Dinleyici** İçindir.

```
final controller = StreamController<int>();

controller.stream.listen((data) {
  print('Dinleyici 1: $data');
});
```

Bu Tür Stream'e İkinci Bir Dinleyici Eklemek Uygun Değildir.

Broadcast Stream

Aynı Akışı Birden Fazla Yer Dinleyecekse Broadcast Stream Kullanılır.

```dart
import 'dart:async';

void main() {
  final controller = StreamController<int>.broadcast();

  controller.stream.listen((data) {
    print('UI Ekranı: $data');
  });

  controller.stream.listen((data) {
    print('Analytics Servisi: $data');
  });

  controller.add(10);

  controller.close();
}
```

Çıktı:

```
UI Ekranı: 10
Analytics Servisi: 10
```

Bir Diğer Yöntem:

```dart
final controller = StreamController<int>();

final broadcastStream = controller.stream.asBroadcastStream();
```

> Broadcast Stream'de Veri Gönderildiği Anda Aktif Dinleyici Yoksa, O Veri Sonradan Bağlanan Dinleyiciye Otomatik Olarak Tekrar Gönderilmez.

6. Abstract

Bir İşlemin **Nasıl Yapıldığını Gizleyip** Uygulamanın Hangi Özellikleri Sunduğunu

Tanımlamaktır.

Örneğin Giriş İşlemi İçin Bir Sözleşme:

```
abstract class AuthRepository {
  Future<String> login(String email, String password);
  Future<void> logout();
  bool get isAuth;
}
```

Bu Sınıf Şunları Garanti Eder:

- Giriş Yapılabilir.
- Çıkış Yapılabilir.
- Giriş Durumu Öğrenilebilir.

Fakat İşlemin Firebase, REST API veya Yerel Veriyle Nasıl Yapılacağını Belirtmez.

Gerçek ve Test Uygulamaları

Firebase Kullanan Sınıf:

```
class FirebaseAuthRepository implements AuthRepository {
  @override
  Future<String> login(String email, String password) async {
    return 'firebase_token_xyz_123';
  }

  @override
  Future<void> logout() async {
    // Firebase Çıkış İşlemi.
  }

  @override
  bool get isAuth => true;
}
```

Test İçin Sahte Sınıf:

```dart
class MockAuthRepository implements AuthRepository {
  @override
  Future<String> login(String email, String password) async {
    await Future.delayed(const Duration(seconds: 1));
    return 'mock_token_abc_789';
  }

  @override
  Future<void> logout() async {}

  @override
  bool get isAuth => false;
}
```

Bu Yaklaşım Sayesinde Uygulamanın Geri Kalanı Veri Kaynağına Doğrudan Bağımlı Olmaz. Test Yazmak Ve Veri Kaynağını Değiştirmek Kolaylaşır.

7. Mixin

Bir Sınıfa Kalıtım Zinciri Oluşturmadan Ek Davranış Kazandırır.

```dart
mixin Logger {
  void log(String message) {
    print('LOG: $message');
  }
}
```

Kullanımı:

```dart
class PaymentService with Logger {
  void pay() {
    log('Ödeme Tamamlandı');
  }
}
```

Bir Sınıfa Birden Fazla Mixin Eklenebilir:

```
mixin Logger {
  void log(String message) => print('LOG: $message');
}

mixin Analytics {
  void log(String message) => print('ANALYTICS: $message');
}

class PaymentService with Logger, Analytics {}
```

Burada:

```
PaymentService().log('Ödeme tamamlandı');
```

Çıktısı:

```
ANALYTICS: Ödeme tamamlandı
```

Çünkü Aynı İsimli Metotlarda Sağ Tarafta Bulunan Mixin Öncelik Kazanır.

on ile Mixin Sınırlandırmak

```
abstract class BaseWidget {}

mixin LifeLogger on BaseWidget {
  void logInit() {
    print('Widget başlatıldı');
  }
}
```

Bu Mixin Yalnızca `BaseWidget` Sınıfından Türetilen Sınıflarda Kullanılabilir.

8. Extension

Extension, Hazır Bir Sınıfın Kodunu Değiştirmeden Ona Yeni Özellik Eklemeyi Sağlar.

```
extension StringValidator on String {
  bool get isValidEmail {
    final emailRegex = RegExp(
      r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$',
    );

    return emailRegex.hasMatch(this);
  }

  String toCapitalize() {
    if (isEmpty) return this;

    return this[0].toUpperCase() + substring(1);
  }
}
```

Kullanımı:

```
void main() {
  const email = 'user@example.com';

  print(email.isValidEmail); // true
  print('mustafa'.toCapitalize()); // Mustafa
}
```

9. Generic Yapılar

Generic Yapılar, Aynı Kodun Farklı Veri Türleriyle Güvenli Biçimde Çalışmasını Sağlar.

```
class Cache<T> {
  final Map<String, T> storage = {};
}
```

Kullanım Örnekleri:

```
final stringCache = Cache<String>();
final intCache = Cache<int>();
```

Generic Kısıtlama

Bir Generic Türün Belirli Bir Özelliğe Sahip Olmasını İsteyebilirsin:

```
abstract class Identifiable {

String get id;

}

class User implements Identifiable {

@override

final String id;

final String name;

User({

required this.id,

required this.name,

});

}
```

```
class GenericCache<T extends Identifiable> {

final Map<String, T> _storage = {};

void save(T item) {

// item'ın id Özelliği Olduğu Kesin Oldığı için Kullanılabilir.

_storage[item.id] = item;

}

T? getById(String id) {

// id varsa Nesneyi, Yoksa null Döndürür.

return _storage[id];

}

}

void main() {

final userCache = GenericCache<User>();

final user = User(

id: 'user-1',

name: 'Mustafa',

);

userCache.save(user);

final foundUser = userCache.getById('user-1');

print(foundUser?.name); // Mustafa

}
```

10. Flutter Widget Tree

Flutter'daki Her Arayüz Parçası Bir Widget'tır.

```
MaterialApp(
  home: Scaffold(
    appBar: AppBar(
      title: Text('Başlık'),
    ),
    body: Center(
      child: Text('Merhaba'),
    ),
  ),
);
```

Bu Yapının Widget Tree Görünümü:

```
MaterialApp
 └── Scaffold
      ├── AppBar
      │    └── Text
      └── Center
           └── Text
```

Widget'ların Özellikleri:

- **Immutable:** Oluşturulduktan Sonra İçlerindeki Değerler Değiştirilmez.
- **Hafif:** Sık Oluşturulmaları Normaldir.
- **Tariftir:** Ekranın Nasıl Görünmesi Gerektiğini Açıklar.
- **Doğrudan Çizim Yapmaz:** Yerleşim ve Çizim İşlemlerini Başka Flutter Katmanları Gerçekleştirir.

11. Element Tree ve RenderObject

Flutter Arayüzü Kabaca Üç Katmanla Düşünülebilir:

Katman	Görevi
Widget	Arayüzün Değişmez Tarifidir.
Element	Widget'ı Çalışma Zamanında Yönetir ve Ağaca Bağlar.
RenderObject	Ölçme, Yerleştirme ve Ekrana Çizme İşlerini Yapar.

Element Tree:

- Widget'ların Ekrandaki Yaşayan Karşılığıdır.
- Değişebilir (`mutable`) Yapıdadır.
- Widget Güncellemelerini Takip Eder.
- `StatefulWidget` İçindeki `State` Nesnesini Korur ve Yönetir.

Her Widget'ın Doğrudan Bir RenderObject'i Olmak Zorunda Değildir; Bazı Widget'lar Yalnızca Diğer Widget'ları Düzenler.

12. `canUpdate` , Key ve State Koruma

Flutter, Yeni Widget Geldiğinde Eski Element'in Kullanılıp Kullanılamayacağını Kontrol Eder.

Temel Mantık:

```
static bool canUpdate(Widget oldWidget, Widget newWidget) {
  return oldWidget.runtimeType == newWidget.runtimeType &&
      oldWidget.key == newWidget.key;
}
```

Yani:

- Widget Türü Aynıysa,
- `key` Değeri Aynıysa,

Flutter Mevcut Element'i ve Bağlı `State` Nesnesini Kullanmaya Devam Edebilir.

Bu Nedenle `Key` , Özellikle Liste Elemanlarında Önemlidir.

```
ListTile(
  key: ValueKey(user.id),
  title: Text(user.name),
);
```

Liste Sıralaması Değiştiğinde Flutter, Hangi State'in Hangi Kullanıcıya Ait Olduğunu Daha Doğru Takip Eder.

13. StatefulWidget Yaşam Döngüsü

`StatefulWidget` , Zaman İçinde Değişebilen Arayüzler İçin Kullanılır.

Örnekler:

- Sayaç,
- Form,
- API'den Gelen Veriler,
- Tema Değişimi,
- Yükleniyor Durumu,
- Animasyonlar.

Temel Yaşam Döngüsü:

```
createState()
   ↓
initState()
   ↓
didChangeDependencies()
   ↓
build()
   ↓
didUpdateWidget() → build()
   ↓
deactivate()
   ↓
dispose()
```

createState()

State Nesnesini Oluşturur.

```dart
@override
State<MyPage> createState() => _MyPageState();
```

initState()

Widget İlk Kez Oluşturulurken Yalnızca Bir Kez Çağrılır.

```dart
@override
void initState() {
  super.initState();

  // Controller Oluşturma.
  // İlk Veri İsteğini Başlatma.
  // Stream Dinlemeye Başlama.
}
```

`didChangeDependencies()`

İlk Kurulumda `initState()` Sonrasında Çağrılır. Bağımlı Olunan Değerler Değişirse Tekrar Çalışabilir.

```
@override
void didChangeDependencies() {
  super.didChangeDependencies();

  final theme = Theme.of(context);
}
```

`Theme.of(context)` ve `MediaQuery.of(context)` Gibi Context Bağımlı Yapılar Burada Kullanılabilir.

`build()`

Ekranın Güncel Görünümünü Tarif Eder.

```
@override
Widget build(BuildContext context) {
  return const Scaffold(
    body: Center(
      child: Text('Merhaba Flutter'),
    ),
  );
}
```

`build()` Birden Fazla Kez Çağrılabilir. Bu Nedenle İçinde:

- API Çağrısı Başlatmamak,
- Stream Aboneliği Oluşturmamak,
- Controller Üretmemek,
- Ağır Hesaplama Yapmamak

Gerekir.

didUpdateWidget()

Aynı Element'e Yeni Widget Yapılandırması Geldiğinde Çağrılır.

```
@override
void didUpdateWidget(covariant MyWidget oldWidget) {
  super.didUpdateWidget(oldWidget);

  // Eski ve Yeni Değerleri Karşılaştırabilirsin.
}
```

Örneğin Dışarıdan Gelen Bir `Stream` veya `Controller` Değişirse Eski Bağlantıyı Kapatıp Yenisini Başlatmak İçin Kullanılabilir.

deactivate()

Widget Ağaçtan Geçici Olarak Çıkarıldığında Çalışır. Widget Aynı Kare İçinde Başka Konuma Taşınabilir. Bu Yüzden Genel Temizlik İşlemleri İçin Genellikle Doğru Yer Değildir.

dispose()

State Kalıcı Olarak Kaldırılırken Bir Kez Çağrılır.

```
@override
void dispose() {
  controller.dispose();
  subscription.cancel();

  super.dispose();
}
```

Burada Kapatılması Gereken Yaygın Kaynaklar:

- `TextEditingController`
- `AnimationController`
- `ScrollController`

- `FocusNode`
- `StreamSubscription`
- Timer
- Diğer Dinleyici Ve Controller Yapıları

`mounted` Kontrolü

Asenkron İşlem Sürerken Kullanıcı İlgili Ekrandan Ayrılmış Olabilir. Bu Durumda İşlem Bittikten Sonra `setState()` Çağırmak Hata Oluşturabilir.

```
Future<void> loadData() async {
  final data = await fetchData();

  if (!mounted) return;

  setState(() {
    // Ekranı gelen veriyle güncelle
  });
}
```

`mounted` , State Nesnesinin Hâlâ Widget Ağacında Olup Olmadığını Belirtir.

14. Flutter Geliştirme Ortamı Kurulumu

15. İlk Flutter Projesini Oluşturma

```
flutter create test
```

Buradaki `test` , Proje Adıdır. Proje Adlarında:

- Türkçe Karakter,
- Boşluk,

- Büyük Harf,
- Özel Karakter

Kullanılmamalıdır.

Proje Oluşturulduktan Sonra Proje Klasörüne Girilir:

```
cd test
```

Flutter Uygulamasını Çalıştırmak İçin:

```
flutter run
```

Komutu Kullanılır.

Belirli Bir Cihazda Çalıştırmak İçin `-d` Parametresi Kullanılabilir.

Örneğin Chrome Üzerinde Çalıştırmak İçin:

```
flutter run -d chrome
```

Bu Komut İlk Çalıştırmada Gerekli Web Dosyalarını İndirebilir. Uygulama Başarıyla Açılıyorsa Flutter Kurulumu Çalışır Durumdadır.

16. Varsayılan Flutter Proje Yapısı

Yeni Oluşturulan Flutter Projesinde En Önemli Dosyalar Şunlardır:

```
proje_adi/
 ├── lib/
 │    └── main.dart
 ├── pubspec.yaml
 ├── android/
 ├── ios/
 ├── web/
 ├── test/
 └── pubspec.lock
```

Dosya / Klasör	Görevi
`lib/main.dart`	Uygulamanın Başlangıç Kodlarının Bulunduğu Dosyadır.
`pubspec.yaml`	Proje Bilgileri, SDK Sürümleri ve Paket Bağımlılıklarını İçerir.
`pubspec.lock`	Kurulan Paketlerin Kesin Sürümlerini Tutar.
`android/`	Android'e Özel Yapılandırmaları İçerir.
`ios/`	iOS'a Özel Yapılandırmaları İçerir.
`web/`	Web Platformuna Özel Dosyaları İçerir.

17. `main.dart` ve Uygulama Başlangıcı

Flutter Uygulaması `main()` Fonksiyonundan Başlar.

```dart
void main() {
  runApp(const MyApp());
}
```

Burada:

- `main()` : Uygulamanın İlk Çalışan Fonksiyonudur.
- `runApp()` : Verilen Widget'ı Uygulamanın Kök Widget'ı Olarak Çalıştırır.
- `MyApp` : Uygulamanın Ana Widget'ıdır.

Flutter Arayüzleri Widget'lardan Oluşur. Her Görsel Yapı, Metin, Buton, Sayfa veya Yerleşim Elemanı Bir Widget'tır.

18. `StatelessWidget` ile Temel Uygulama

Değişmeyen Arayüzler İçin `StatelessWidget` Kullanılır.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const DtmMasterApp());
}

class DtmMasterApp extends StatelessWidget {
  const DtmMasterApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'DTM Flutter Sınıfı',
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6366F1),
        ),
      ),
      home: const LiveCanvasScreen(),
    );
  }
}
```

Bu Yapıda:

- `StatelessWidget` : İç Durumu Değişmeyen Widget Türüdür.
- `build()` : Ekranda Gösterilecek Widget Ağacını Döndürür.

- `MaterialApp` : Material Design Tabanlı Uygulamanın Ana Yapısıdır.
- `debugShowCheckedModeBanner` : Sağ Üstteki Debug Yazısını Gizler.
- `title` : Uygulama Başlığıdır.
- `theme` : Uygulamanın Genel Tema Ayarlarını İçerir.
- `home` : Uygulama İlk Açıldığında Gösterilecek Sayfadır.

Widget'lar Değişmez Yapıda Olduğu İçin Uygun Yerlerde `const` Kullanılması Performans Açısından Faydalıdır.

```
const DtmMasterApp({super.key});
```

Burada:

- `const` : Widget'ın Değişmeyeceğini Belirtir.
- `key` : Flutter'ın Widget'ları Ayırt Etmesine Yardımcı Olan Kimlik Bilgisidir.
- `super.key` : `key` Değerini Üst Sınıfa Aktarır.

19. `Scaffold`, `AppBar` ve `body`

Bir Sayfanın Temel Material Yapısı Genellikle `Scaffold` ile Kurulur.

```dart
class LiveCanvasScreen extends StatelessWidget {
  const LiveCanvasScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text(
          'DTM Canlı Arayüz',
          style: TextStyle(
            fontWeight: FontWeight.bold,
            fontSize: 18,
          ),
        ),
        centerTitle: true,
        backgroundColor: Colors.white,
        elevation: 0,
        actions: [
          IconButton(
            onPressed: () {},
            icon: const Icon(Icons.notifications_outlined),
          ),
        ],
      ),
      body: const Center(
        child: Text('Buraya Canlı Arayüz Gelecek'),
      ),
    );
  }
}
```

Bu Yapıda:

Widget	Görevi
`Scaffold`	Sayfanın Temel Ekran Yapısını Kurar.
`AppBar`	Sayfanın Üst Başlık Alanıdır.
`body`	Sayfanın Ana İçerik Alanıdır.
`Center`	Altındaki Widget'ı Ortalar.
`Text`	Ekrana Metin Yazdırır.
`IconButton`	Tıklanabilir İkon Butonu Oluşturur.
`actions`	AppBar'ın Sağ Tarafına Birden Fazla Widget Eklemeyi Sağlar.

> `actions` Liste Yapısındadır. Çünkü AppBar'a Birden Fazla Buton veya İkon Eklenebilir.

20. `pubspec.yaml`

`pubspec.yaml` , Flutter Projesinin Yapılandırma Dosyasıdır. Proje Adı, Açıklaması, Sürümü, SDK Gereksinimleri ve Kullanılan Paketler Bu Dosyada Tanımlanır.

Temel Örnek:

```yaml
name: dtm_project
description: DTM 15 Ekim Projesi
publish_to: none
version: 1.0.0+1

environment:
  sdk: ">=3.0.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true
```

Önemli Alanlar:

Alan	Görevi
`name`	Proje Adını Belirtir.
`description`	Projenin Açıklamasıdır.
`publish_to: none`	Projenin Pub.dev Üzerinde Yayınlanmayacağını Belirtir.
`version`	Uygulama Sürümünü Tanımlar.
`environment`	Kullanılacak Dart SDK Sürüm Aralığını Belirtir.
`dependencies`	Uygulama Çalışırken Gerekli Paketleri İçerir.
`dev_dependencies`	Test, Kod Analizi ve Geliştirme Sürecindeki Paketleri İçerir.
`uses-material-design`	Material İkonlarının Kullanılabilmesini Sağlar.

`pubspec.yaml` Dosyasında Değişiklik Yapıldıktan Sonra Paketlerin İndirilmesi Gerekir.

Terminalde Proje Klasörü İçindeyken Şu Komut Çalıştırılır:

```
flutter pub get
```

Bu Komut Sonrasında `pubspec.lock` Dosyası Oluşur veya Güncellenir.

> Terminal Komutları Çalıştırılmadan Önce Doğru Proje Klasöründe Olunduğundan Emin Olunmalıdır.

21. Flutter Hatalarını Okuma ve Çözme

Flutter Geliştirirken Hata Almak Normaldir. Önemli Olan Hata Mesajını Okuyup Hatanın Kaynağını Bulabilmektir.

Örnek Hatalar:

```
colorScheme
```

`ColorScheme` Bir Sınıf Olduğu İçin Büyük Harfle Yazılmalıdır.

```
actions: [
  ...
]
```

`actions` , Birden Fazla Widget Alabildiği İçin Liste Olarak Tanımlanmalıdır.

```
useMaterial3: true
```

Material 3 Kullanımı İçin Özellik Adı Doğru Yazılmalıdır.

```
Scaffold(
  appBar: AppBar(...),
  body: Center(...),
)
```

`body` , `AppBar` İçinde Değil; `Scaffold` İçinde Tanımlanmalıdır.

Hata Çözümünde İzlenecek Sıra

Bir Hata Alındığında Şu Sıra İzlenebilir:

1. Terminaldeki İlk Hata Mesajını Oku.
2. Hatanın Gösterdiği Dosya ve Satır Numarasına Git.
3. Büyük/Küçük Harf, Virgül, Parantez ve Widget Hiyerarşisini Kontrol Et.
4. `pubspec.yaml` Değiştiyse `flutter pub get` Çalıştır.
5. Flutter Ortamında Sorun Varsa `flutter doctor -v` Çalıştır.
6. VS Code Flutter'ı Tanımıyorsa VS Code'u Yeniden Başlat.
7. PATH İçinde Birden Fazla Flutter Kurulumu Olup Olmadığını Kontrol Et.
8. Proje Klasöründe Olduğundan Emin Ol.